

Pourquoi React ? Avantages et cas d'usage

Objectifs de cette section

Avant de plonger dans le code, il est essentiel de comprendre **pourquoi** on utilise React.

Vous allez découvrir les **problèmes que React résout**, ses **forces principales** et dans quels contextes il est particulièrement utile.

À la fin de cette section, vous serez capable de :

- Identifier les limites du DOM manuel et des Web Components.
- Comprendre les bénéfices du modèle déclaratif.
- Définir les cas d'usage où React est pertinent.
- Expliquer les grandes idées qui sous-tendent React.

1. Les limites du DOM manuel

Dans une application complexe, manipuler le DOM à la main devient vite :

- **Verbeux** : il faut tout écrire soi-même.
- **Fragile** : on oublie de mettre à jour un élément, ou on le fait deux fois.
- **Imprévisible** : une action modifie l'interface, mais on ne sait plus trop comment.

Les Web Components améliorent ça, mais ne permettent pas :

- La **gestion automatique de l'état**.
- Le **rendu conditionnel** et **synchronisé**.
- La **communication fluide** entre composants.

Exemple : comparaison React vs Vanilla JS

```
// Vanilla JS (manipulation manuelle)
const div = document.getElementById("status");
if (isLoggedIn) {
  div.textContent = "Bienvenue !";
} else {
  div.textContent = "Veuillez vous connecter.";
}
```

```
// React (déclaratif)
function Status({isLoggedIn}) {
  return <p>{isLoggedIn ? "Bienvenue !" : "Veuillez vous connecter."}</p>;
}
```

2. React : une bibliothèque pour créer des interfaces

React est une **bibliothèque JavaScript** développée par Facebook.

Elle permet de créer des interfaces **déclaratives, modulaires et réactives**.

Ses fondements :

- L'interface est une fonction de l'état ($UI = f(state)$).
- Chaque composant possède ses propres **props** et **state**.
- Lorsqu'un état change, **React reconstruit virtuellement l'interface**, puis met à jour uniquement ce qui a changé dans le DOM réel.

Exemple : $UI = f(state)$

```
function Compteur() {
  const [count, setCount] = React.useState(0);
  return (
    <div>
      <p>Compteur : {count}</p>
      <button onClick={() => setCount(count + 1)}>Incrémenter</button>
    </div>
  );
}
```

Ici, la valeur affichée dépend uniquement du state, et React s'occupe de tout re-rendre automatiquement.

3. Les avantages concrets de React

- **Déclaratif** : on décrit "ce qu'on veut voir", pas "comment le modifier".
- **Modulaire** : on construit une interface en composants réutilisables.
- **Performant** : grâce au **virtual DOM**, React limite les manipulations réelles.
- **Écosystème riche** : outils, extensions, communauté.

React simplifie notamment :

- Les **applications mono-page** (SPA).
- Les interfaces dynamiques avec **état évolutif**.
- Les **formulaires complexes**, les **interfaces temps réel**, etc.

4. Quand React est-il pertinent ?

React est utile lorsque :

- Vous construisez une **interface interactive**, avec des mises à jour fréquentes.
- L'application a **plusieurs composants** qui doivent **communiquer** ou réagir aux mêmes données.
- Vous avez besoin d'une structure claire et maintenable sur le long terme.

5. Exercice de mise en pratique

Consignes

1. Rédigez un court document (ou présentation) qui répond aux questions suivantes :
 - Quels problèmes React permet-il de résoudre par rapport à Vanilla JS ?
 - Quelles sont les **3 caractéristiques principales** de React ?
 - Citez **2 situations concrètes** où l'usage de React est recommandé.
2. Préparez un fichier de projet vide (`App.jsx`) qui servira de base pour la section suivante.

Organisation attendue

```
/exercices
├── 10-intro-react
│   ├── App.jsx
│   └── main.jsx
```